

```

/*
 * C program to Implement Radix Sort
 */

#include<stdio.h>

#include<stdlib.h>

int *Radix_sort(int *arr, int size);

int *Count_sort(int *arr, int size, int Exponent);

int maximum(int *arr,int length);

int minimum(int *arr,int length);

int main()

{

    int i, size;

    int *arr;

    printf("Enter the array size: ");

    scanf("%d",&size);

    arr = (int *) malloc( sizeof( int ) * size );

    if(arr==NULL)

    {

        exit(-1);//abnormal termination.

    }

    else

    {

        // Entering the array values

        printf("Enter the array\n");

```

```

    for(i = 0; i < size; i++)
    {
        printf("arr[ %d ] = ",i);

        scanf("%d",&arr[i]);
    }

    printf("Array before sorting:\n");

    for(i = 0; i < size; i++)
    {
        printf("arr[%d] = %d\n",i,arr[i]);
    }


    arr = Radix_sort(arr,size);
}

printf("ARRAY AFTER SORTING: \n");

for(int i=0;i<size;i++)
{
    printf("arr[ %d ] = %d \n",i ,arr[i]);
}
}

```

```

int *Radix_sort(int *arr, int size)
{
    int Max_of_arr = maximum(arr, size);

    int Exponent = 1;

    int count = 0;

```

```

while(Exponent <= Max_of_arr)
{
    arr = Count_sort(arr, size, Exponent);

    Exponent= Exponent* 10;

    //uncomment the loop to see how sorting happens digit after moving from LSB to MSB.

    /*

        printf("ARRAY AFTER SORTING: %d digits from rightmost element\n",count);

        for(int i=0;i<size;i++)

            printf("arr[ %d ] = %d \n",i ,arr[i]);

        count++;

    */

}

return arr;
}

```

```

int *Count_sort(int *arr, int size, int Exponent)
{
    int range = 10;

    int *frequency_array ;

    frequency_array = (int*)malloc(sizeof(int)* range);

    if(frequency_array == NULL)
    {
        exit(-1);
    }

    int sum=0;

```

```
for(int i=0; i<range; i++)
```

```
{
```

```
    frequency_array[ i ]=0;
```

```
}
```

```
for(int i=0;i<size;i++)
```

```
{
```

```
    frequency_array[ (arr[i]/Exponent)%10 ]++;
```

```
}
```

```
for(int i =0; i<range;i++)
```

```
{
```

```
    sum = sum + frequency_array[i];
```

```
    frequency_array[i] = sum;
```

```
}
```

```
int *new_arr;//new array to store the result.
```

```
new_arr = (int *)malloc(sizeof(int) *size);
```

```
if(new_arr == NULL)
```

```
{
```

```
    exit(-1);
```

```
}
```

```
else
```

```
{
```

```
    int pos;
```

```

    for(int i=size-1; i>=0 ;i-- )
    {
        pos = frequency_array[(arr[i]/Exponent)%10]-1;
        new_arr[ pos ] = arr[ i ];
        frequency_array [(arr[i]/Exponent)%10]--;
    }
}

return new_arr;
}

int maximum(int *arr, int length)
{
    int max=INT_MIN;
    for( int i=0 ; i<length ; i++ )
    {
        if(arr[i]>max)
            max=arr[i];
    }
    return max;
}

int minimum(int *arr, int length)
{
    int min=INT_MAX;
    for( int i=0 ; i<length ; i++ )
    {
        if(arr[i]<min)

```

```
        min=arr[i];  
    }  
    return min;  
}
```